

Examen Final — Práctica CI/CD: Documento de Reflexión

nnyez

29 de julio de 2026

1. Justificación de la estrategia de despliegue (Canary)

Se eligió **Canary** sobre Blue-Green por las siguientes razones técnicas, aplicadas al caso concreto de `inventario-app`:

- **Proporcionalidad nativa de Services:** Kubernetes distribuye el tráfico de forma proporcional a la cantidad de pods que coinciden con el selector del Service. Con 4 réplicas estables (v1, azul) y 1 réplica canary (v2, verde), el ~80% del tráfico va a la versión estable y ~20% a la nueva, sin necesidad de herramientas externas como Argo Rollouts.
- **Riesgo mínimo:** Dado que `inventario-app` es una API REST con base de datos JSON local, un error en la nueva versión solo afectaría al ~20% de los usuarios, permitiendo detectar problemas sin comprometer toda la aplicación.
- **Infraestructura eficiente:** Canary requiere solo 1 réplica extra (5 pods total) frente a las 4 réplicas duplicadas que necesitaría Blue-Green (8 pods total). Para una aplicación ligera como esta, la diferencia es significativa.
- **Rollback inmediato:** Para revertir, basta con escalar el Deployment canary a 0 réplicas (`kubectl scale deployment/inventario-app-canary --replicas=0`).

2. Observación sobre persistencia de datos

Se creó un producto nuevo (“Producto nuevo”, SKU: NEW-001) vía `POST /api/products`, confirmando que el catálogo contenía 4 productos. Posteriormente se forzó la eliminación de un pod:

```
kubectl delete pod -n inventario-app -l app=inventario-app --limit=1
```

Tras la recreación del pod, el nuevo producto había desaparecido: el catálogo contenía solo los 3 productos seed originales.

Causa: La base de datos es un archivo JSON local (`data/products.json`) almacenado en el sistema de archivos efímero del pod. Cuando Kubernetes recrea el pod, utiliza la imagen original que contiene únicamente los 3 productos seed. No hay un PersistentVolumeClaim (PVC) ni una base de datos externa.

Conclusión: No es un error. Es el comportamiento esperado de una base de datos local sin persistencia externa. Para un entorno productivo, se necesitaría un PVC o una base de datos externa (PostgreSQL, MySQL, etc.).

3. Problemas reales encontrados

Problema	Causa	Solución
Pipeline fallaba en build-push	La acción <code>aquasecurity/trivy-action@v0.36.0</code> no existe (las versiones usan formato <code>v0.x.x</code>)	Cambiar a <code>homebrew/trivy-action@v0.36.0</code>
Pipeline fallaba en Build and push	<code>cache-to: type=gha</code> no es compatible con el driver <code>docker</code> de <code>buildx</code>	Eliminar la configuración de caché
Error “denied: installation not allowed”	El <code>GITHUB_TOKEN</code> por defecto no tiene permiso <code>packages: write</code>	Agregar <code>permissions: packages: write</code> al workflow
Trivy encontraba CRITICALs y fallaba el pipeline	<code>node:18-alpine</code> incluía <code>libcrypto3</code> (CVE-2026-31789) y <code>npm</code> bundled con <code>tar@6.2.1</code> (CVE-2026-59873)	Cambiar a <code>node:18-alpine3.21</code> , agregar <code>apk upgrade</code> , y eliminar <code>npm</code> del runtime
Pods no arrancaban en Minikube	El Deployment referenciaba <code>inventario-app-secrets</code> que no existía	Crear el Secret con <code>kubectl create secret</code>
Service no balanceaba tráfico correctamente	<code>kubectl port-forward</code> sobre el Service no distribuye tráfico entre pods	Probar desde dentro del clúster con <code>kubectl run</code>

4. Métricas DORA

Lead Time for Changes

Tiempo entre el commit y el despliegue en el clúster (`kubectl apply`):

Cambio	Commit Timestamp	Despliegue Timestamp	Lead Time
Pipeline base (8cd4a8b)	2026-07-29 04:26	2026-07-29 04:35	~9 min
Canary + Secret (112578a)	2026-07-29 05:06	2026-07-29 05:07	~1 min
STARTUP_DEPLOY (2d25c12)	2026-07-29 05:07	2026-07-29 05:08	~1 min

Deployment Frequency

Métrica	Valor
Total despliegues (pipelines ejecutados)	10
Pipelines exitosos	5
Días de trabajo	1 (2026-07-29)
Frecuencia	10 deploys/día

Change Failure Rate

Métrica	Valor
Total despliegues	10
Fallos (pipelines en rojo)	5
Tasa de fallo	50%

Reflexión: La alta tasa de fallo (50%) refleja la etapa inicial de configuración del pipeline, donde se corrigieron problemas de autenticación, versión de acciones, caché de Docker y vulnerabilidades de seguridad. Según la tabla de niveles DORA vista en clase, una tasa de fallo del 50% corresponde al nivel **“Bajo” (Low)**, típico de equipos que están estableciendo su pipeline por primera vez. Con un pipeline maduro, se espera reducir esta tasa por debajo del 15% (nivel “Alto”).